

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_87c49086-be4\agent-a6e2e66ae746ebe7f.jsonl`
- SHA-256: `d8461a51ecd1c59a9caa965354aa864eacb9e44fba559b06b48d6f44116fa061`
- Source modified: `2026-06-18T08:13:04+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_87c49086-be4`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T08:05:22.323Z)

You implement ONE behavior-preserving refactor and open a PR for OWNER REVIEW (do NOT merge). You are in your OWN isolated git worktree of the badminton-highlight-indexer repo on THIS machine.
The Python env has pytest/ruff/mypy/cv2/torch/numpy ? the full offline suite RUNS here (ignore any CLAUDE.md note saying it can't).

SETUP FIRST, exactly:

```
git fetch origin
git checkout -B <BRANCH> origin/master      # base MUST be origin/master, NOT the current HEAD
then verify: `git log --oneline -1` is origin/master's HEAD and `git branch --show-current` == <BRANCH>.
```

RULES (pure refactor ? quality must NOT regress, behavior must NOT change):

- Behavior-PRESERVING moves/extractions ONLY. Do NOT change logic, branch conditions, log strings/levels, return types, ordering, thresholds, or constants. It is cut-paste into helpers.
- Edit ONLY the files named in the task. Stage EXPLICIT paths (`git add <file>`), NEVER -A/./-u.
- Match the surrounding code style and naming.

GATE (binding ? run all four; capture pass/skip counts + coverage %):

```
python run_all_tests.py
python -m ruff check .
python -m mypy backend training
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
```

ONLY IF ALL FOUR ARE GREEN:

```
git add <explicit files>
git commit -m "<conventional commit msg>"   # final line: Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
git push -u origin <BRANCH>
gh pr create --base master --head <BRANCH> --title "<title>" --body "<concise body: what/why, behavior-preserving argument, gate results>" # NO merge
IF ANY GATE IS RED: do NOT push or open a PR. Fix if it's a trivial slip you introduced;
otherwise
report status='gate-failed' with the failing output and your diff so far. Never weaken a test or lower the coverage floor to go green.
```

Return the structured object with: slug, branch, status ('pr-opened'|'gate-failed'|'error'), pr_url (""+reason if none), diffstat, suite (e.g. '851 passed, 7 skipped'), ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

=== YOUR TASK: tracknet-extract (branch refactor/tracknet-dedup-and-active-frames) ===
FILE: backend/pipeline/detectors/tracknet_runner.py. TWO behavior-preserving changes:

(A) DEDUP: the two-line "find group index nearest the temporal midpoint and chop" logic (``mid=(g[0][0]+g[-1][0])/2.0; split_i=min(range(1,len(g)), key=lambda i: abs(g[i][0]-mid))``) is copy-pasted in the W1 hard-cap fallback (in `_split_overlong_groups`) and in `_blob_fallback_chop`. Add a private `@staticmethod _midpoint_split_index(group) -> int`` (body = those two lines verbatim, returning the index) and replace BOTH inline copies with a call. (B) EXTRACT: in `trajectory_to_action_windows`, lift the opening per-frame velocity loop that builds ``active = [(frame_idx, velocity)]`` into `@staticmethod _active_frames(points, fps, frame_width, velocity_thresh) -> List[Tuple[int,float]]``; keep the `fps<=0/frame_width<=0` defaulting in the public method BEFORE the call so identical floats are fed in, then ``if not active: return []`` and the rest unchanged. Output must be element-for-element identical. TESTS: `tests/test_tracknet_runner.py` (frame-set partition invariant + hard-cap/blob-fallback splits), `tests/test_windowing.py` (preset windows). This is ONE PR covering both small changes (same file).

You are running in an isolated git worktree at `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-5` (a separate working copy of the repo). Changes you make here do NOT affect the main working directory (`C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\refactor-base`) or other agents. Work normally ? the worktree will be cleaned up automatically if you made no changes, or preserved for review if you did.

2. assistant (2026-06-18T08:05:24.323Z)

I'll start by setting up the worktree on the correct base branch.

3. user (2026-06-18T08:05:29.109Z)

branch 'refactor/tracknet-dedup-and-active-frames' set up to track 'origin/master'.
Switched to a new branch 'refactor/tracknet-dedup-and-active-frames'
`a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)`
---BRANCH---
refactor/tracknet-dedup-and-active-frames

4. assistant (2026-06-18T08:05:31.875Z)

Branch is set up correctly on origin/master HEAD (a010667). Now let me read the target file.

5. user (2026-06-18T08:05:33.304Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
```

```

22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                        # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                    # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                      # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                    # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                    # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p
77
78

```

```

79     @dataclass
80     class TrajectoryPoint:
81         frame: int
82         visible: bool
83         x: float
84         y: float
85
86
87     class TrackNetRunner(WslCommandMixin, DetectorRunner):
88         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90         Implements the common DetectorRunner contract so a future Linux-native or
91         alternative trajectory runner can be substituted without touching the hybrids.
92         """
93
94         def __init__(self, cfg: TrackNetConfig):
95             self.cfg = cfg
96
97         def healthcheck(self) -> Tuple[bool, str]:
98             """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100            Returns (ok, message). Cheap to call before a long run.
101            """
102            cmd = (
103                f"conda activate {self.cfg.conda_env} && "
104                f"python -c \"import tensorflow as tf; "
105                f"print('TF', tf.__version__, 'GPUs', "
106                f"len(tf.config.list_physical_devices('GPU'))))\" "
107            )
108            try:
109                res = self._wsl_bash(cmd)
110            except FileNotFoundError:
111                return False, "wsl` not found ? is this running on Windows with WSL2
112                installed?"
113            except subprocess.TimeoutExpired:
114                return False, "WSL healthcheck timed out."
115            out = (res.stdout or "").strip()
116            if res.returncode != 0:
117                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118            return True, out
119
120            # ----- inference
121
122            def run_predict(self, video_win_path: str, output_win_dir: str,
123                video_id: Optional[str] = None) -> Optional[str]:
124                """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125                None.
126
127                ``output_win_dir`` is a Windows directory (under the repo's output/) that
128                WSL writes into via its /mnt view, so the Windows process can read the CSV
129                back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130                therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131                a filename cannot collide on the output CSV.
132                """
133                if not self.cfg.weights_path:
134                    logger.error(
135                        "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136                        "in config.json to the WSL path of your trained TrackNetV4 weights."
137                    )
138                return None
139
140                # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
141                # relative paths through unchanged, so WSL would resolve them against the
142                # predict.py cwd instead of the repo (same bug class as wasb_runner).
143                output_win_dir = os.path.abspath(output_win_dir)

```

```

141         os.makedirs(output_win_dir, exist_ok=True)
142         # Normalize separators so basename/splitext work when tests (or collector)
143         # pass Windows-style paths on a Linux host.
144         video_win_path = str(video_win_path).replace("\\", "/")
145         base = os.path.basename(video_win_path)
146         stem, ext = os.path.splitext(base)
147
148         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149         if ext.lower() not in (".mp4", ".avi"):
150             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151             return None
152
153         wsl_out = to_wsl_mnt_path(output_win_dir)
154         # Namespace by video_id so two same-filename videos don't collide on the CSV.
155         # predict.py derives its output name from the (staged) video stem, so staging
156         # under the namespaced name propagates into "<key>_predict.csv".
157         key = f"{stem}_{video_id[:12]}" if video_id else stem
158         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160         # Resolve the video path WSL will read.
161         if self.cfg.stage_in_wsl:
162             staged = self._stage_video(video_win_path, f"{key}{ext}")
163             if staged is None:
164                 return None
165             wsl_video = staged
166         else:
167             # No staging: predict.py reads /mnt directly and writes
168             # "<stem>_predict.csv".
169             # We cannot rename its output, so fall back to the un-namespaced name.
170             wsl_video = to_wsl_mnt_path(video_win_path)
171             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173         if self.cfg.min_free_gb > 0:
174             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175             if free is not None and free < self.cfg.min_free_gb:
176                 logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                               "consider running tools/cleanup_caches.py.",
178                               free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180         cmd = (
181             f"conda activate {self.cfg.conda_env} && "
182             f"cd {self.cfg.repo_dir}/src && "
183             f"python predict.py "
184             f"--video_path {wsl_tilde_quote(wsl_video)} "
185             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186             f"--output_dir {wsl_tilde_quote(wsl_out)} "
187             f"--queue_length {self.cfg.queue_length}"
188         )
189         logger.info("Running TrackNet inference on %s ...", base)
190         try:
191             res = self._wsl_bash(cmd)
192         except subprocess.TimeoutExpired:
193             logger.error("TrackNet inference timed out after %ss.",
194                         self.cfg.timeout_sec)
195             return None
196
197         if res.returncode != 0:
198             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
199                     res.stdout)[-2000:])
200             return None
201
202         if not os.path.exists(expected_csv_win):
203             logger.error("TrackNet finished but expected CSV not found at %s",
204                         expected_csv_win)
205             return None

```

```

202         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205         # regenerable intermediate). On earlier failure we return above without
cleaning.
206         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207             logger.info("Cache hygiene: removing staged video for %s "
208                         "(set indexing.tracknet.keep_frames=true to retain).", key)
209             self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
224         {wsl_tilde_quote(dest)} && echo OK"
225         try:
226             res = self._wsl_bash(cmd)
227         except subprocess.TimeoutExpired:
228             logger.error("Staging video into WSL timed out.")
229             return None
230         if res.returncode != 0 or "OK" not in (res.stdout or ""):
231             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
232             return None
233             return dest
234
235     # ----- CSV -> windows
236
237     @staticmethod
238     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
239         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
240         points: List[TrajectoryPoint] = []
241         with open(csv_win_path, newline="") as f:
242             reader = csv.DictReader(f)
243             for row in reader:
244                 try:
245                     points.append(TrajectoryPoint(
246                         frame=int(row["Frame"]),
247                         visible=int(row["Visibility"]) == 1,
248                         x=float(row["X"]),
249                         y=float(row["Y"]),
250                     ))
251                 except (KeyError, ValueError):
252                     continue
253             points.sort(key=lambda p: p.frame)
254             return points
255
256     @staticmethod
257     def trajectory_to_action_windows(
258         points: List[TrajectoryPoint],
259         fps: float,
260         frame_width: float,
261         chunk_start: float = 0.0,
262         velocity_thresh: float = 0.02,
263         merge_gap: float = 2.0,
264         min_window_duration: float = 2.0,

```

```

264         chunk_index: Optional[int] = None,
265         max_window_duration: float = 0.0,
266         min_split_gap: float = 0.5,
267         window_hard_cap: bool = False,
268         window_inactivity_end: float = 0.0,
269         inactivity_min_density: float = 0.34,
270         inactivity_rally_thresh: float = 0.08,
271         window_blob_fallback: bool = False,
272         window_blob_factor: float = 4.0,
273     ) -> List[Dict[str, Any]]:
274         """Convert a shuttle trajectory into candidate rally windows.
275
276         A frame is "active" when the shuttle is visible AND its inter-frame
277         displacement (normalised by frame width, per second) exceeds
278         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
279         ``merge_gap`` seconds of each other are grouped into one window; short
280         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
281         shape feeding the AI-handoff phase is identical.
282
283         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
284         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
285         progress" and a single window swallows several rallies + the dead time between them. When
286         set, any window longer than this is recursively SPLIT at its largest internal inactivity
287         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
288         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
289         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
290         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
291
292         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
293         an over-long window has NO internal inactivity gap to split on (a continuously-
294         active trajectory ? the shuttle never stops moving, e.g. the real-footage
295         ``mahadevpura-1`` blob where the gap-splitter alone left a single 174 s window), it is recursively
296         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
297         cuts (never merges), so recall cannot drop; gated default-OFF until measured.
298
299         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
300         unlike ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
301         every gap-less over-cap window and over-segments normal clips), this runs AFTER the R4
302         inactivity trim and force-chops ONLY a genuine *blob* ? a group still longer than
303         ``window_blob_factor`` x ``max_window_duration`` once the gap-split and the principled rally-end trim
304         have both had their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
305         is left alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
306         logged, and each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
307         that needs rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
308         (recall can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until

```

```

measured.
309         """
310         if fps <= 0:
311             fps = 30.0
312         if frame_width <= 0:
313             frame_width = 1280.0
314
315         # Compute per-frame normalised velocity from the last *visible* point.
316         active = [] # (frame_idx, velocity)
317         prev: Optional[TrajectoryPoint] = None
318         for p in points:
319             if not p.visible:
320                 prev = None # break the trajectory; absent shuttle = not in flight
321                 continue
322             if prev is not None:
323                 dframes = p.frame - prev.frame
324                 if dframes > 0:
325                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
326                     velocity = (dist / frame_width) * (fps / dframes)
327                     if velocity > velocity_thresh:
328                         active.append((p.frame, velocity))
329             prev = p
330
331         if not active:
332             return []
333
334         max_gap_frames = int(merge_gap * fps)
335
336         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337             vels = [v for _, v in group]
338             return {
339                 "start": group[0][0] / fps + chunk_start,
340                 "end": group[-1][0] / fps + chunk_start,
341                 "active_frame_count": len(group),
342                 "peak_velocity": max(vels),
343                 "mean_velocity": sum(vels) / len(vels),
344                 "track_id_count": 1, # single shuttle; kept for window-shape parity
345                 "chunk_index": chunk_index,
346             }
347
348         groups: List[List[Tuple[int, float]]] = []
349         group = [active[0]]
350         for af in active[1:]:
351             if af[0] - group[-1][0] <= max_gap_frames:
352                 group.append(af)
353             else:
354                 groups.append(group)
355                 group = [af]
356         groups.append(group)
357
358         if max_window_duration > 0:
359             groups = TrackNetRunner._split_overlong_groups(
360                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
361                 hard_cap=window_hard_cap)
362
363         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
366         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
367         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.

```

```

368         if window_inactivity_end > 0:
369             tw = int(window_inactivity_end * fps)
370             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
371                                                         inactivity_rally_thresh) for g
in groups]
372
373         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
374         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
375         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
376         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
377         forced_flags = [False] * len(groups)
378         if window_blob_fallback and max_window_duration > 0:
379             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
380                 groups, int(max_window_duration * fps), window_blob_factor)
381
382         windows = []
383         for g, forced in zip(groups, forced_flags):
384             w = _finalize(g)
385             if forced:
386                 w["forced_cut"] = True
387             windows.append(w)
388         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
389
390     @staticmethod
391     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
392                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
393         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
394         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
395         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
396         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
397         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
398         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
399         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
400         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
401         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
402         if trail_frames <= 0 or len(group) < 2:
403             return group
404         frames = [f for f, _ in group]
405         fast = [1 if v > rally_thresh else 0 for _, v in group]
406         lo = 0
407         fast_sum = 0
408         last_dense = -1
409         for i in range(len(frames)):
410             fast_sum += fast[i]
411             while frames[lo] <= frames[i] - trail_frames:
412                 fast_sum -= fast[lo]
413                 lo += 1
414             win_count = i - lo + 1
415             if win_count > 0 and fast_sum / win_count >= min_density:
416                 last_dense = i
417         return group[: last_dense + 1] if last_dense >= 0 else group
418

```

```

419     @staticmethod
420     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
421                               min_split_frames: int,
422                               hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
423         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
424         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
425         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
426         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
427         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
428         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
429         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
430         if max_win_frames <= 0:
431             return groups
432         out: List[List[Tuple[int, float]]] = []
433         work = list(groups)
434         while work:
435             g = work.pop()
436             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
437                 out.append(g)
438                 continue
439             best_i, best_gap = -1, -1
440             for i in range(1, len(g)):
441                 gap = g[i][0] - g[i - 1][0]
442                 if gap > best_gap:
443                     best_gap, best_i = gap, i
444             if best_i < 0 or best_gap < min_split_frames:
445                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
446                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
447                 if hard_cap:
448                     mid = (g[0][0] + g[-1][0]) / 2.0
449                     split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
450                     work.append(g[:split_i])
451                     work.append(g[split_i:])
452                 else:
453                     out.append(g)
454                     continue
455                 work.append(g[:best_i])
456                 work.append(g[best_i:])
457             out.sort(key=lambda grp: grp[0][0])
458         return out
459
460     @staticmethod
461     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
462                             blob_factor: float = 4.0
463                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
464         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
465         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
466         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
467         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
468         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)

```

```

469         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
470         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
471         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
472         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
473         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
474         that piece came from a forced chop. Iterative worklist to bound stack depth."""
475         if max_win_frames <= 0:
476             return groups, [False] * len(groups)
477         blob_frames = max_win_frames * max(1.0, blob_factor)
478         out: List[List[Tuple[int, float]]] = []
479         flags: List[bool] = []
480         forced = 0
481         for g in groups:
482             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
483                 out.append(g)
484                 flags.append(False)
485                 continue
486             pieces: List[List[Tuple[int, float]]] = []
487             work = [g]
488             while work:
489                 h = work.pop()
490                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
491                     pieces.append(h)
492                     continue
493                 mid = (h[0][0] + h[-1][0]) / 2.0
494                 split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) #
noqa: B023
495                 work.append(h[:split_i])
496                 work.append(h[split_i:])
497             pieces.sort(key=lambda grp: grp[0][0])
498             out.extend(pieces)
499             flags.extend([True] * len(pieces))
500             forced += len(pieces) - 1
501         if forced:
502             logger.warning(
503                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
504                 "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
505                 "motion), not a bigger cap", forced)
506         return out, flags
507

```

6. assistant (2026-06-18T08:05:37.124Z)

I have the full file. Now let me make the three edits (one staticmethod helper for dedup A, two replacements, and the extraction B).

First, let me add the `\_midpoint\_split\_index` staticmethod and replace the two inline copies. Let me place the new helper near the other static helpers. I'll add it right before `\_split\_overlong\_groups`.

7. user (2026-06-18T08:05:42.295Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-5/backend/pipeline/detectors/tracknet_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

8. assistant (2026-06-18T08:05:45.205Z)

Now replace the inline copy in `\_split\_overlong\_groups`:

9. user (2026-06-18T08:05:47.360Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-5/backend/pipeline/detectors/tracknet_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

10. assistant (2026-06-18T08:05:49.877Z)

Now replace the inline copy in `\_blob\_fallback\_chop`:

11. user (2026-06-18T08:05:52.154Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-5/backend/pipeline/detectors/tracknet_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

12. assistant (2026-06-18T08:05:55.258Z)

Now part B ? extract the active-frames loop into `\_active\_frames`. The defaulting (`fps <= 0`, `frame\_width <= 0`) must stay BEFORE the call in the public method. Let me make the extraction.

13. user (2026-06-18T08:05:59.591Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-5/backend/pipeline/detectors/tracknet_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

14. assistant (2026-06-18T08:06:01.970Z)

Now add the `\_active\_frames` staticmethod. I'll place it right after `parse\_trajectory\_csv` and before `trajectory\_to\_action\_windows`. Let me view the current state around there to find a clean anchor.

15. user (2026-06-18T08:06:04.302Z)

```
252         points.sort(key=lambda p: p.frame)
253         return points
254
255     @staticmethod
256     def trajectory_to_action_windows(
257         points: List[TrajectoryPoint],
258         fps: float,
259         frame_width: float,
```

16. user (2026-06-18T08:06:14.064Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-5/backend/pipeline/detectors/tracknet_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

17. assistant (2026-06-18T08:06:17.917Z)

Let me view the modified `trajectory\_to\_action\_windows` region to confirm the comment placement and that nothing was left dangling.

18. user (2026-06-18T08:06:18.788Z)

```
329         their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
330         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
```

```

331         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334         """
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:
338             frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]

```

19. assistant (2026-06-18T08:06:21.677Z)

All edits are in place. The logic is element-for-element identical: the defaulting stays before the call, the loop body is verbatim, and both midpoint-split copies now call the shared helper with identical math.

Now let me run the gate. First the test suite.

20. user (2026-06-18T08:08:26.088Z)

```

=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
.....SS..... [ 8%]
..... [ 16%]
..... [ 25%]
..... [ 33%]
..... [ 41%]
..... [ 50%]
..... [ 58%]
..... [ 66%]
..... [ 75%]
.....SSSSS..... [ 83%]
..... [ 91%]
..... [100%]
856 passed, 7 skipped in 118.57s (0:01:58)
=====
RESULT: PASS - all tests green
=====

```

21. assistant (2026-06-18T08:08:28.122Z)

Suite is green: 856 passed, 7 skipped. Now ruff and mypy.

22. user (2026-06-18T08:08:29.798Z)

All checks passed!

23. user (2026-06-18T08:09:45.916Z)

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 104 source files
```

24. assistant (2026-06-18T08:09:52.231Z)

mypy clean (only the default notes, no issues). Now the coverage gate.

25. user (2026-06-18T08:11:58.507Z)

backend\tools__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	104	47	55%
backend\utils__init__.py	0	0	100%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	27	3	89%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\video.py	25	3	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	108	11	90%

TOTAL	8016	1544	81%

Required test coverage of 70% reached. Total coverage: 80.74%
856 passed, 7 skipped, 7 warnings in 119.94s (0:01:59)
=====

RESULT: PASS - all tests green

=====

26. assistant (2026-06-18T08:12:02.209Z)

All four gates green: 856 passed / 7 skipped, ruff clean, mypy clean, coverage 80.74% (?70 floor). Let me confirm coverage of the touched file specifically and check the diff before committing.

27. user (2026-06-18T08:12:04.093Z)

```
backend/pipeline/detectors/tracknet_runner.py | 55 ++++++-----
1 file changed, 36 insertions(+), 19 deletions(-)
===VERIFY HEAD/BRANCH===
refactor/tracknet-dedup-and-active-frames
```

28. assistant (2026-06-18T08:12:07.864Z)

Only the one named file changed, and `origin/master..HEAD` is empty (no sibling commits ride along). Let me view the full diff one more time to be sure it's a pure move.

29. user (2026-06-18T08:12:08.880Z)

```
diff --git a/backend/pipeline/detectors/tracknet_runner.py
b/backend/pipeline/detectors/tracknet_runner.py
index dd1785b..9f03c49 100644
--- a/backend/pipeline/detectors/tracknet_runner.py
+++ b/backend/pipeline/detectors/tracknet_runner.py
@@ -252,6 +252,31 @@ class TrackNetRunner(WslCommandMixin, DetectorRunner):
     points.sort(key=lambda p: p.frame)
```

```

        return points

+   @staticmethod
+   def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
+       velocity_thresh: float) -> List[Tuple[int, float]]:
+       """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in flight.
+
+       Computes each visible point's normalised velocity from the last *visible* point;
+       an absent shuttle breaks the trajectory. A frame qualifies when its velocity exceeds
+       ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already defaulted by
+       the caller so identical floats feed the velocity math."""
+       active = [] # (frame_idx, velocity)
+       prev: Optional[TrajectoryPoint] = None
+       for p in points:
+           if not p.visible:
+               prev = None # break the trajectory; absent shuttle = not in flight
+               continue
+           if prev is not None:
+               dframes = p.frame - prev.frame
+               if dframes > 0:
+                   dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
+                   velocity = (dist / frame_width) * (fps / dframes)
+                   if velocity > velocity_thresh:
+                       active.append((p.frame, velocity))
+               prev = p
+       return active

+   @staticmethod
+   def trajectory_to_action_windows(
+       points: List[TrajectoryPoint],
@@ -312,21 +337,7 @@ class TrackNetRunner(WslCommandMixin, DetectorRunner):
    if frame_width <= 0:
        frame_width = 1280.0

-   # Compute per-frame normalised velocity from the last *visible* point.
-   active = [] # (frame_idx, velocity)
-   prev: Optional[TrajectoryPoint] = None
-   for p in points:
-       if not p.visible:
-           prev = None # break the trajectory; absent shuttle = not in flight
-           continue
-       if prev is not None:
-           dframes = p.frame - prev.frame
-           if dframes > 0:
-               dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
-               velocity = (dist / frame_width) * (fps / dframes)
-               if velocity > velocity_thresh:
-                   active.append((p.frame, velocity))
-       prev = p
+   active = TrackNetRunner._active_frames(points, fps, frame_width, velocity_thresh)

    if not active:
        return []
@@ -416,6 +427,14 @@ class TrackNetRunner(WslCommandMixin, DetectorRunner):
    last_dense = i
    return group[: last_dense + 1] if last_dense >= 0 else group

+   @staticmethod
+   def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
+       """Index of the active frame nearest the group's temporal midpoint (used to chop a
+       gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
+       fallback."""
+       mid = (group[0][0] + group[-1][0]) / 2.0
+       split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
+       return split_i

```

```

+
    @staticmethod
    def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames: int,
                               min_split_frames: int,
@@ -445,8 +464,7 @@ class TrackNetRunner(WslCommandMixin, DetectorRunner):
    # Too long but no real dead-time gap. Default: keep as one (a genuine long
    rally).

    # hard_cap: chop at the temporal midpoint so the cap is actually enforced.
    if hard_cap:
-        mid = (g[0][0] + g[-1][0]) / 2.0
-        split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
+        split_i = TrackNetRunner._midpoint_split_index(g)
        work.append(g[:split_i])
        work.append(g[split_i:])
    else:
@@ -490,8 +508,7 @@ class TrackNetRunner(WslCommandMixin, DetectorRunner):
    if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
        pieces.append(h)
        continue
-        mid = (h[0][0] + h[-1][0]) / 2.0
-        split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) # noqa: B023
+        split_i = TrackNetRunner._midpoint_split_index(h)
        work.append(h[:split_i])
        work.append(h[split_i:])
    pieces.sort(key=lambda grp: grp[0][0])

```

30. assistant (2026-06-18T08:12:13.615Z)

The diff is a pure move. Note the `# noqa: B023` was on a line that no longer contains the lambda capture (the lambda now lives in `\_midpoint\_split\_index` where its loop variable `i` is a comprehension-local `min(range(...), key=lambda i: ...)`), not a closure over a loop variable? so no B023, confirmed by ruff passing). All gates green. Let me commit, push, and open the PR.

31. user (2026-06-18T08:12:20.537Z)

```

[refactor/tracknet-dedup-and-active-frames 79bd7f1] refactor(tracknet): dedup midpoint-split +
extract _active_frames helper
 1 file changed, 36 insertions(+), 19 deletions(-)
===COMMITTED===
79bd7f1 refactor(tracknet): dedup midpoint-split + extract _active_frames helper

```

32. user (2026-06-18T08:12:26.274Z)

```

remote:
remote: Create a pull request for 'refactor/tracknet-dedup-and-active-frames' on GitHub by
visiting:
remote:   https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/tracknet-
dedup-and-active-frames
remote:
branch 'refactor/tracknet-dedup-and-active-frames' set up to track 'origin/refactor/tracknet-
dedup-and-active-frames'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      refactor/tracknet-dedup-and-active-frames -> refactor/tracknet-dedup-and-
active-frames

```

33. user (2026-06-18T08:12:44.557Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/222>

34. assistant (2026-06-18T08:12:46.348Z)

PR opened. Let me confirm its state.

35. user (2026-06-18T08:12:49.217Z)

```
{"baseRefName":"master","headRefName":"refactor/tracknet-dedup-and-active-frames","number":222,"state":"OPEN","title":"refactor(tracknet): dedup midpoint-split + extract _active_frames helper"}
```

36. assistant (2026-06-18T08:12:52.248Z)

PR #222 is open against master (not merged), as required. All four gates passed. Returning the structured result.

37. user (2026-06-18T08:13:02.752Z)

Structured output provided successfully

38. assistant (2026-06-18T08:13:04.839Z)

Done. Refactor complete, all four gates green, PR #222 opened against master for owner review (not merged).